
3.13 MHA-MasterFailover 分析

作者：王向

前言

MHA 出来将近 10 年的时间，作为一个开源产品，能活这么久，还有这么多人追捧。基本上可以说是一种 mysql 中的标准解决方案。

不过 MHA 已经不适合这个时代了。。。但是这不影响我们对他进行一波研究。

发生 master crash 源码分析

下面的内容比较淦，没有兴趣的同学直接跳到：总结。

关于源码可以直接去 github.com 然后搜 MHA。

```
sub main {
    .....
    eval { $error_code = do_master_failover(); };
    if ($?) {
        $error_code = 1;
    }
    if ($error_code) {
        finalize_on_error();
    }
    return $error_code;
    ....
}

sub do_master_failover {
    my $error_code = 1; #错误码
    my ( $dead_master, $new_master );
    # $dead_master 去世的 master
    # $new_master 新生的 master
    eval {
        # 第一步： 检查配置
        my ( $servers_config_ref, $binlog_server_ref ) = init_config();
        $log->info("Starting master failover.");
        $log->info();
        $log->info("* Phase 1: Configuration Check Phase..\n");
        $log->info();
    }
}
```

```

# 初始化 Binlog server
MHA::ServerManager::init_binlog_server( $binlog_server_ref, $log );
    # ssh_check_simple 检查 SSH 的连通性
    # get_node_version 获取 node 的版本号
    # 实际使用命令拿到版本号 apply_diff_relay_logs --version
    # 拿到版本号 Binlog server 可达，否则不可达
$dead_master = check_settings($servers_config_ref);
    # MHA::ManagerUtil::check_node_version($log); # 检查 mha 的版本信息
        # 结果 1: 没有安装 mha
        # 结果 2: $node_version
< $MHA::ManagerConst::NODE_MIN_VERSION
        # our $NODE_MIN_VERSION = '0.54';
        # 节点版本号必须等于或者高于 0.54
    # $_server_manager->connect_all_and_read_server_status(); # 检查各实例是否可以
连接
    # my @dead_servers = $_server_manager->get_dead_servers();
    # my @alive_servers = $_server_manager->get_alive_servers();
    # my @alive_slaves = $_server_manager->get_alive_slaves();
    # get_dead_servers/get_alive_servers/get_alive_slaves: double check 各个 node 的
死活状态

    # g$_server_manager->start_sql_threads_if(); 查看 Slave_SQL_Running 是否为 Yes,
若不是则启动 SQL thread

if ( $_server_manager->is_gtid_auto_pos_enabled() ) {
    $log->info("Starting GTID based failover.");
}
else {
    $_server_manager->force_disable_log_bin_if_auto_pos_disabled();
    $log->info("Starting Non-GTID based failover.");
}
$log->info();
$log->info("** Phase 1: Configuration Check Phase completed.\n");
$log->info();

# 第二步: 关闭当前失败的 IO 复制线程，并执行脚本切换 VIP
$log->info("** Phase 2: Dead Master Shutdown Phase..\n");
$log->info();

force_shutdown($dead_master);

# 判断 ssh 是否可达
    # MHA::HealthCheck::ssh_check_simple()

```

```

        # MHA::ManagerUtil::get_node_version()
    # my $rc = $target->stop_io_thread(); 停止所有 slave 复制 IO 线程
    # force_shutdown_internal() 执行配置文件中的
master_ip_failover_script/shutdown_script, 如果没有就不执行
    # master_ip_failover_script: 如果设置了 VIP, 则首先切换 VIP
    # shutdown_script: 如果设置了 shutdown 脚本, 则执行 shutdown 脚本

$log->info("* Phase 2: Dead Master Shutdown Phase completed.\n");
$log->info();

# 第三步 新主恢复
$log->info("* Phase 3: Master Recovery Phase..\n");
$log->info();

# 获取新的主从信息
$log->info("* Phase 3.1: Getting Latest Slaves Phase..\n");
$log->info();
check_set_latest_slaves();
    # $_server_manager->read_slave_status(); 获取各个 slave 的 binlog file 和
position 点
    # my %status = $target->check_slave_status(); 执行 show slave status 来获取从
库信息

        Slave_IO_State,
        Master_Host,
        Master_Port,
        Master_User,
        Slave_IO_Running,
        Slave_SQL_Running,
        Master_Log_File,
        Read_Master_Log_Pos,
        Relay_Master_Log_File,
        Last_Errno,
        Last_Error,
        Exec_Master_Log_Pos,
        Relay_Log_File,
        Relay_Log_Pos,
        Seconds_Behind_Master,
        Retrieved_Gtid_Set,
        Executed_Gtid_Set,
        Auto_Position,
        Replicate_Do_DB,
        Replicate_Ignore_DB,
        Replicate_Do_Table,
        Replicate_Ignore_Table,

```

```
Replicate_Wild_Do_Table,  
Replicate_Wild_Ignore_Table
```

```
#其中重要的信息
```

```
$target->{Relay_Master_Log_File} = $status{Relay_Master_Log_File};
```

```
$target->{Exec_Master_Log_Pos} = $status{Exec_Master_Log_Pos};
```

```
$target->{Relay_Log_File} = $status{Relay_Log_File};
```

```
$target->{Relay_Log_Pos} = $status{Relay_Log_Pos};
```

```
# $_server_manager->identify_latest_slaves(); 比较各个 slave 的 Master_Log_File  
和 Read_Master_Log_Pos,寻找 latest 的 slave
```

```
# $_server_manager->identify_oldest_slaves(); 比较各个 slave 中的  
Master_Log_File 和 Read_Master_Log_Pos,寻找 Oldest 的 slave
```

```
if ( !$_server_manager->is_gtid_auto_pos_enabled() ) {
```

```
    $log->info();
```

```
    # 进行 binlog 补充
```

```
    $log->info("* Phase 3.2: Saving Dead Master's Binlog Phase..\n");
```

```
    $log->info();
```

```
    save_master_binlog($dead_master);
```

```
    # 判断 dead master 是否可以 ssh 连接
```

```
    # if ( $_real_ssh_reachable && !$g_skip_save_master_binlog ) {
```

```
        # 如果 dead master 可以 ssh 连接
```

```
        # MHA::ManagerUtil::check_node_version();
```

```
        # my $latest_pos = ( $_server_manager->get_latest_slaves() )[0]-  
>{Read_Master_Log_Pos}; save_master_binlog_internal( $latest_file, $latest_pos, $dead_m  
aster, );
```

```
        # 使用 node 节点的 save_binary_logs 脚本在 dead master 上做拷贝
```

```
        # save_binary_logs --command=save --start_file=$master_log_file --  
start_pos=$read_master_log_pos --binlog_dir=$dead_master->{master_binlog_dir} --  
output_file=$diff_binary_log_remote --handle_raw_binlog=$dead_master-  
>{handle_raw_binlog} --disable_log_bin=$dead_master->{disable_log_bin} --  
manager_version=$MHA::ManagerConst::VERSION";
```

```
        # generate_diff_binary_log():
```

```
        # $_binlog_manager-  
>concat_all_binlogs_from($start_binlog_file, $start_binlog_pos, $out_diff_file)
```

```
        # dump_binlog() 拷贝 binlog 文件到到 manage 节点的 manager_workdir 目录下, 如果  
dead master 无法 ssh 登录, 则 master 上未同步到 slave 的 txn 丢失  
    }
```

```
$log->info();
```

```
# 确定新主
```

```
$log->info("* Phase 3.3: Determining New Master Phase..\n");
```

```
$log->info();
```



```

my $latest_base_slave;
if ( $_server_manager->is_gtid_auto_pos_enabled() ) {
    $latest_base_slave = $_server_manager->get_most_advanced_latest_slave();
}
else {
    $latest_base_slave = find_latest_base_slave($dead_master); #寻找最新的有所有中继日
志的 slave, 用于恢复其他 slave
    # my $latest_base_slave = find_latest_base_slave_internal();

    # my $oldest_mlf      = $oldest_slave->{Master_Log_File};
    # my $oldest_mlp      = $oldest_slave->{Read_Master_Log_Pos};
    # my $latest_mlf      = $latest_slaves[0]->{Master_Log_File};
    # my $latest_mlp      = $latest_slaves[0]->{Read_Master_Log_Pos};

    # if ($_server_manager-
>pos_cmp( $oldest_mlf, $oldest_mlp, $latest_mlf,$latest_mlp ) >= 0 # 判断 latest 和
oldest slave 上 binlog 位置是不是相同, 相同就不需要同步 relay log
    # apply_diff_relay_logs --command=find --latest
    # 查看 latest slave 中是否有 oldest 缺少的 relay log, 若无则继续, 否则 failover 失败
    # 查找的方法: 逆序的读 latest slave 的 relay log 文件, 一直找到 binlog file 的
position 为止
    }
    $new_master = select_new_master( $dead_master, $latest_base_slave ); #选出新的
master 节点
    # 比较 master_log_file:read_master_log_pos
    # 识别优先从库, 在线的并带有 candidate_master 标记
    # 识别应该忽略的从库, 带有 no_master 标记、或者未开启 log_bin、与最新从库相比数据延
迟比较大(slave 与 master 的 binlog position 差距大于 100000000)
    # 选择优先级依次为: 优先列表、最新从库列表、所有从库列表, 但一定排除忽略列表
    # 检查新老主库的复制过滤规则是否一致
Replicate_Do_DB,Replicate_Ignore_DB,Replicate_Do_Table,Replicate_Ignore_Table

my ( $master_log_file, $master_log_pos, $exec_gtid_set ) =
    recover_master( $dead_master, $new_master, $latest_base_slave,
    $binlog_server_ref );
$new_master->{activated} = 1;

$log->info(" * Phase 3: Master Recovery Phase completed.\n");
$log->info();
# 恢复从库 类似单独恢复主库的过程
$log->info(" * Phase 4: Slaves Recovery Phase..\n");
$log->info();
$error_code = recover_slaves(

```

```

    $dead_master,    $new_master,    $latest_base_slave,
    $master_log_file, $master_log_pos, $exec_gtid_set
); # 中继补偿（生成 Slave 与 New Slave 之间的差异日志，将该日志拷贝到各 Slave 的工作目录
下），指向新主库&启动复制（change_master_and_start_slave），清理新主库的 slave 复制通道
(reset slave all)

if ( $g_remove_dead_master_conf && $error_code == 0 ) {
    MHA::Config::delete_block_and_save( $g_config_file, $dead_master->{id},
    $log );
}
cleanup();
};
if ($@) {
    if ( $dead_master && $dead_master->{not_error} ) {
        $log->info($@);
    }
    else {
        MHA::ManagerUtil::print_error( "Got ERROR: $@", $log );
        $mail_body .= "Got Error so couldn't continue failover from here.\n"
        if ($mail_body);
    }
    $_server_manager->disconnect_all() if ($_server_manager);
    undef $@;
}
eval {
    send_report( $dead_master, $new_master );
    MHA::NodeUtil::drop_file_if( $_status_handler->{status_file} )
    unless ($error_code);

    if ($_create_error_file) {
        MHA::NodeUtil::create_file_if($_failover_error_file);
    }
};
if ($@) {
    MHA::ManagerUtil::print_error( "Got ERROR on final reporting: $@", $log );
    undef $@;
}
return $error_code;
}

```

总结

切换过程：

1. 检查配置

- 检查 mha node 的节点版本信息（get_node_version，实际使用 apply_diff_relay_logs --version 命令）
- 检查所有 node 节点的 SSH 的连通性
- 检查所有 node 节点的存活状态
- 检查所有从库的 slave sql 线程是否已经启动，没有启动则启动

2. 关闭当前失败的 IO 复制线程，并执行脚本切换 VIP

- 检查所有 node 节点的 SSH 的连通性
- 停止所有从库的 slave io 线程，只要有一个在线从库的 slave io 线程停止失败，那么就终止切换
- 执行 master_ip_failover_script，保证崩溃主库所在主机的 VIP 存活防脑裂，并执行脚本切换 VIP

3. 新主恢复

- 获取新的主从信息
 - 获取各个 slave 的复制信息（show slave status）
 - 获取复制延迟最小的从库、复制延迟最大的从库
 - 如果没有用来补偿的基准从库，终止切换
- 检查去世的 master 的 SSH 的连通性
 - 不可达，就会丢失 binlog
 - 可达，执行 save_binary_logs --command=save，将保存后的 binlog 拷贝到 manage 节点的 manager_workdir 目录下

4. 选择新主库

- 比较 master_log_file:read_master_log_pos
- 识别优先从库，在线的并带有 candidate_master 标记
- 识别应该忽略的从库，带有 no_master 标记、或者未开启 log_bin、与最新从库相比数据延迟比较大（slave 与 master 的 binlog position 差距大于 100000000）
- 选择优先级依次为：优先列表、最新从库列表、所有从库列表，但一定排除忽略列表
- 检查新老主库的复制过滤规则是否一致 Replicate_Do_DB,Replicate_Ignore_DB,Replicate_Do_Table,Replicate_Ignore_Table
- 恢复新主库
 - 新主库落后于最新从库，那么 ssh 连接上最新从库，执行 apply_diff_relay_logs --command=generate_and_send
 - 等待新主库上已经有的 relaylog 都重放完毕，停止 slave sql 线程
 - 执行主控机上的 master_ip_failover --command=start 脚本，激活新主库的 VIP
 - 关闭新主库的只读，开启可写模式

5. 恢复从库 & 清理新主库

- 恢复新从库
 - 中继补偿（生成 Slave 与 New Slave 之间的差异日志，将该日志拷贝到各 Slave 的工作目录下）
 - 指向新主库 & 启动复制（`change_master_and_start_slave`）
- 清理新主库的 slave 复制通道（`reset slave all`）

3.14 mysql Federated 引擎最佳实战

作者：马文斌

背景

- 本地 MySQL 数据库要访问远程 MySQL 数据库的表中的数据，又不想用数据同步方式去实现，所以就考虑 FEDERATED 存储引擎来实现。
- 要允许这个存储引擎，当构建 MySQL 时使用 `--with-federated-storage-engine` 来 configure。

Federated 引擎架构：

当客户端发出引用 FEDERATED 表的 SQL 语句时，本地服务器（执行 SQL 语句的地方）和远程服务器（实际存储数据的地方）之间的信息流如下：

1. 存储引擎将遍历 FEDERATED 表具有的每一列，并构造一个引用远程表的适当 SQL 语句。
2. 该语句使用 MySQL 客户端 API 发送到远程服务器。
3. 远程服务器处理该语句，本地服务器检索该语句产生的任何结果（受影响的行数或结果集）。
4. 如果该语句产生结果集，则每一列都将转换为 FEDERATED 引擎期望的内部存储引擎格式，并可用于将结果显示给发出原始语句的客户端。

架构图如下：

